

Lista zadań nr 9

Zadanie 1. (1 pkt)

Pokaż, jaki kod zostanie wygenerowany dla wyrażenia $(2 + 3) * (4 - 1)$ przez kompilator z wykładu (wersja bez akumulatora). Prześledź krok po kroku wykonywanie tego kodu na maszynie wirtualnej.

Zadanie 2. (1 pkt)

Powtórz poprzednie zadanie, ale tym razem dla maszyny wirtualnej z akumulatorem.

Zadanie 3. (2 pkt)

Choć pokazana na wykładzie reprezentacja środowiska jest bardzo prosta, to nie jest ona zbyt efektywna: dostęp do zmiennej wymaga czasu liniowego względem głębokości wyrażenia. Zaproponuj bardziej efektywną reprezentację, opartą na słowniku (moduł `Map`), która pozwala na dostęp w czasie logarytmicznym. By uniknąć przebudowywania całego słownika przy każdym rozszerzaniu środowiska, dla każdej zmiennej pamiętaj jej adres względem dna, a nie wierzchołka stosu, a poprawny adres wyliczaj na podstawie dodatkowej informacji o głębokości stosu.

Zadanie 4. (3 pkt)

Rozszerz implementację kompilatora o konstrukcję `let  $x = e_1$  and  $y = e_2$  in  $e_3$` , która pozwala na definiowanie wielu zmiennych lokalnych. Pamiętaj, że zmienne x i y są dostępne tylko w wyrażeniu e_3 .

Zadanie 5. (2 pkt)

Zauważ, że długie ciągi let-wyrażeń powodują, że kompilator generuje długie ciągi instrukcji MPopN 1. Zmodyfikuj kompilator tak, by na końcu takiego ciągu długości n generował tylko jedną instrukcję MPopN n .

Zadanie 6. (3 pkt)

Dla języka bez let-wyrażeń napisz funkcję odwrotną do kompilacji, która dla danego kodu maszyny wirtualnej odtwarza odpowiadające mu wyrażenie. Zauważ, że taka translacja też będzie miała kształt ewaluatora (maszyny wirtualnej), gdzie wartościami są wyrażenia, a nie liczby całkowite.